Survey Sampling
Project Report: Farms

Bruno PILARCZYK

21401452

Shuyao LI

21708390

M2 Statistics and Econometrics

Toulouse School of Economics

Contents

1 Preliminary analysis **2**
1.1 Defining population and variables . . . 2
1.2 True population total of y calculus . . . 2
1.3 Scatter plot of y on x . . . 2

2 A single sample **3**
2.1 SRSWOR design . . . 3
2.2 Stratified designs 1, 2 and 3 . . . 3
2.2.1 Stratified design 1: Allocation proportional to the sum of x . . . 3
2.2.2 Stratified design 2: Neyman allocation . . . 3
2.2.3 Stratified design 3: Proportional allocation . . . 4
2.2.4 Stratified designs results . . . 4
2.3 Post-stratified estimator . . . 5
2.4 Ratio estimator . . . 5
2.5 Regression estimator . . . 6
2.6 Calibrated estimator . . . 6
2.6.1 Calibration method . . . 7
2.6.2 Calibration result . . . 7

3 Simulations **8**
3.1 Sampling designs comparison . . . 8
3.2 Estimators comparison . . . 10

4 Appendix **15**

1 Preliminary analysis

1.1 Defining population and variables

The *farms* dataset contains the result of US agricultural census conducted in 50 states in 1982, 1987 and 1992. This survey frame contains 2992 counties, hence: the population (let us call it U) is of size $N = 2992$.

- Variable of interest y: `ACRES92`, number of acres dedicated to farms in 1992
- Auxiliary variable x: `ACRES87`, number of acres dedicated to farms in 1987
- Variables for stratification and poststratification[1]: `REGION` and `clust87`.

The categorical variable `clust87` is defined as followed:

$$\texttt{clust87} = \begin{cases} 1 & \text{if } \texttt{LARGEF87} \leq 8 \\ 2 & \text{if } 8 < \texttt{LARGEF87} \leq 27 \\ 3 & \text{if } 27 < \texttt{LARGEF87} \leq 69 \\ 4 & \text{if } \texttt{LARGEF87} \geq 69 \end{cases}$$

Those thresholds are respectively the quartiles of the variable `LARGEF87`, which is number of farms over 1000 acres in 1987. These quartiles are chosen as thresholds as it insures to get homogeneous strata in terms of size, which is a relevant characteristic for post-stratification as it is better to obtain at least one observation from the sample included in each stratum. Otherwise, a stratum would be not included for the estimating totals, variances *etc*, which is not representative.

1.2 True population total of y calculus

We define the true total of the variable of interest y as:

$$t_y = \sum_{i \in U} y_i$$

We obtain $t_y = 923\,355\,862$.

1.3 Scatter plot of y on x

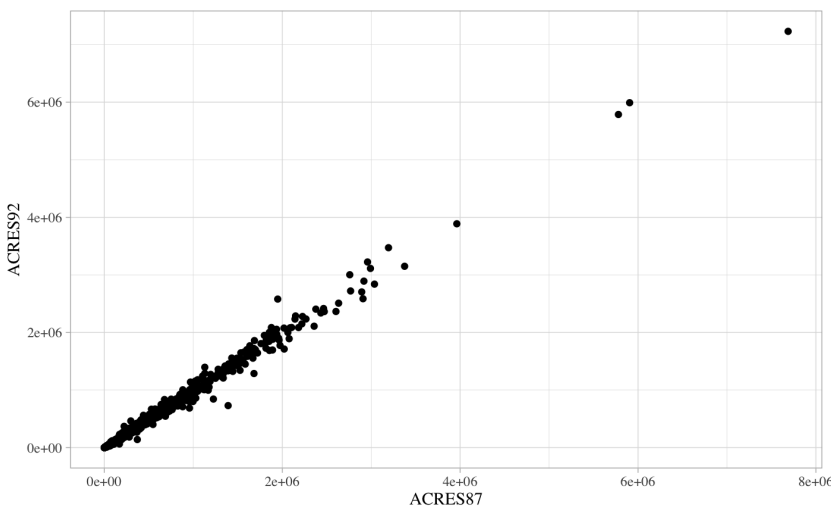

Figure 1: Scatter plot of y on x

[1] `STATE`, the categorical variable indicating the state of the county was not relevant as it would imply 50 strata, which is not relevant when only 80 observations are drawn from the population.

2 A single sample

For this section and the following ones, at each experiment we will draw a sample (let us call it s) of size $n = 80$.

2.1 SRSWOR design

Let us consider $\widehat{t}_y$ the Horvitz-Thomspon estimator of the true total t_y. Furthermore, we consider $\widehat{V}(\widehat{t}_y)$ the variance estimator of the p-unbiased Horvitz-Thompson estimator. In order to understand all formulas, let us define s_y^2 as the sample variance of the total and $\widehat{\bar{t}}_y$ the Horvitz-Thompson estimator of the mean.

Here are the formulas of these estimators:

- $\widehat{t}_y = \dfrac{N}{n}\sum_{i\in s} y_i$
- $\widehat{\bar{t}}_y = \dfrac{1}{n}\sum_{i\in s} y_i$
- $s_y^2 = \dfrac{1}{n-1}\sum_{i\in s}(y_i - \hat{\bar{t}}_y)^2$
- $\widehat{V}_{\text{SRS}}(\widehat{t}_y) = N^2(1 - \dfrac{n}{N})\dfrac{s_y^2}{n}$

We also define the coefficient of variation as $\text{CV}_{\text{SRS}}(\widehat{t}_y) = \dfrac{\sqrt{\widehat{V}(\widehat{t}_y)}}{\widehat{t}_y}$

After computations it results:

- $\widehat{t}_y = 889\,393\,131$
- $\widehat{V}_{\text{SRS}}(\widehat{t}_y) = 1.37e + 16$
- $\text{CV}_{\text{SRS}}(\widehat{t}_y) = 13.15\%$

2.2 Stratified designs 1, 2 and 3

The previous subsection showed how the Horvitz-Thompson estimator behaves in the case of a Simple Random Sampling Without Replacement. In this subsection we will see how it behaves in the case of several stratified designs. Indeed, what will change between the following three different sampling designs is the allocation of the sample ($n = 80$) for each stratum. To get relevant results we will use the variable `clust87` as the stratification variable.

`clust87` $\in \{1, 2, 3, 4\}$. As there are four strata and by construction of the stratification variable, the population is composed as followed: $N_1 = 762, N_2 = 740, N_3 = 747, N_4 = 743$. For example 762 is the number of observations in the first stratum.

Let us define how the allocation of the sample for each stratum will be done before calculating the results.

2.2.1 Stratified design 1: Allocation proportional to the sum of x

x is the auxiliary variable `ACRES87`. We will allocate the sample to each sample stratum as follows:

$$n_h = n \times \frac{\sum_{i\in U_h} x_i}{\sum_{i\in U} x_i} \quad , \quad h \in \{1, 2, 3, 4\}$$

2.2.2 Stratified design 2: Neyman allocation

The Neyman allocation or optimal allocation uses the standard deviation of the true total in each strata in order to allocation the sample. It performs well as using the standard deviation compensate for a high variability of the y-values in each strata.

$$n_h = n \times \frac{N_h S_{y,h}}{\sum_{k=1}^{4} N_k S_{y,k}} \quad , \quad h \in \{1, 2, 3, 4\}$$

$S_{y,h}$ is the standard deviation of the variable of interest over the population strata U_h:

$$S_{y,h} = \sqrt{\frac{1}{N_h - 1} \sum_{i \in U_h} (y_i - \bar{t}_{y,h})^2}$$

Notice that $\bar{t}_{y,h}$ is the true mean of the variable of interest over the population strata U_h,

$$\bar{t}_{y,h} = \frac{t_{y,h}}{N_h} = \frac{1}{N_h} \sum_{i \in U_h} y_i$$

2.2.3 Stratified design 3: Proportional allocation

For this sampling design, each the sample strata has to respect the original proportion of the population in each strata. The proportion of each strata in the population are defined as:

$$\frac{N_h}{N} \quad , \quad h \in \{1, 2, 3, 4\}$$

Hence, it is easy to calculate n_h , $h \in \{1, 2, 3, 4\}$ as it has to respect the following equality:

$$\frac{N_h}{N} = \frac{n_h}{n}$$

We get:

$$n_h = n \times \frac{N_h}{N} \quad , \quad h \in \{1, 2, 3, 4\}$$

2.2.4 Stratified designs results

Here are the formulas for the Horvitz-Thompson estimator (still p-unbiased) and its associated estimated variance in the case of stratified designs:

- $\widehat{t}_{y,\mathrm{ST}} = \sum_{h=1}^{H} \frac{N_h}{n_h} \sum_{i \in s_h} y_i$
- $\widehat{V}_{\mathrm{ST}}(\widehat{t}_{y,\mathrm{ST}}) = \sum_{h=1}^{H} \widehat{V}(\widehat{t}_{y,h})$

The estimator of the variance in the case of stratified designs is the sum of the variance in each sample strata, by independence.

Moreover:

- $\widehat{V}(\widehat{t}_{y,h}) = N_h^2 (1 - \frac{n_h}{N_h}) \frac{s_h^2}{n_h}$
- $s_y^2 = \frac{1}{n_h - 1} \sum_{i \in s_h} (y_i - \hat{\bar{t}}_{y,h})^2$
- $\hat{\bar{t}}_y = \frac{1}{n_h} \sum_{i \in s_h} y_i$

Let us call the stratified sampling designs: allocation proportional to the sum of x, Neyman allocation and proportional allocation; respectively ST1, ST2 and ST3. Here is the matrix summarizing the sample size of each size:

Stratified designs	n_1	n_2	n_3	n_4
ST1	4	10	18	47
ST2	7	7	17	49
ST3	20	20	20	20

Table 1: Sample strata allocation according to the stratified design

We can clearly notice that ST1 and ST2 are made in such a way that they compensate for the large variability in the y-values for the last stratum.

Before the analysis let us define the design effect, which allows to measure the efficiency of a sampling design (not only stratified designs) compared to the SRSWOR design.

$$\mathrm{deff}(design) = \frac{\widehat{V}_{design}(\widehat{t}_y)}{\widehat{V}_{\mathrm{SRS}}(\widehat{t}_y)}$$

Here are the results for the Horvitz-Thompson estimator of the total, the estimation of the variation and the coefficient of variation:

Stratified designs	$\widehat{t}_{y,\mathrm{ST}}$	$\widehat{V}_{\mathrm{ST}}(\widehat{t}_{y,\mathrm{ST}})$	$\mathrm{CV}(\widehat{t}_{y,\mathrm{ST}})$	deff
ST1	837 447 353	3.02e+15	6.56%	22.08%
ST2	884 660 342	4.93e+15	7.93%	35.98%
ST3	838 929 749	7.54e+15	10.35%	55.14%

Table 2: Stratified designs (1,2 and 3) results

We cannot conclude significantly only based on a single sample. However based on these results, all stratified designs are more efficient than the SRSWOR design as $\mathrm{deff}(design) < 1$ for $design \in \{\mathrm{ST1}, \mathrm{ST2}, \mathrm{ST3}\}$

ST1 is the most efficient ($\mathrm{deff}(\mathrm{ST1}) = 22.08\%$) because $\mathrm{deff}(\mathrm{ST1}) < \mathrm{deff}(\mathrm{ST2}) < \mathrm{deff}(\mathrm{ST3})$.

2.3 Post-stratified estimator

In this subsection and in the following ones we will no longer use stratified sampling designs but only the SRSWOR design.

In order to get a post-stratified estimator, first a sample s is drawn from the population U using the SRSWOR design. Afterwards, it is possible to check the sample size in each stratum in order to correct the weights associated to each Horvitz-Thompson estimator within each sample stratum.

Let us defined the post-stratified estimator as followed:

$$\widehat{t}_{\mathrm{post}} = \sum_{h=1}^{H} \widehat{t}_{y,h}$$

Notice that $\widehat{t}_{y,h}$ is the Horvitz-Thompson in each sample stratum:

$$\widehat{t}_{y,h} = \frac{1}{n_h} \sum_{i \in s_h} y_i$$

We will also use this formula for the variance:

$$\widehat{V}(\widehat{t}_{\mathrm{post}}) = \sum_{h=1}^{H} N_h^2 (1 - \frac{n_h}{N_h}) \frac{s_h^2}{n_h}$$

Here are the results for both post-stratified estimators using the post-stratification variables `clust87` and `REGION`:

Post-stratification var.	$\widehat{t}_{\mathrm{post}}$	$\widehat{V}(\widehat{t}_{\mathrm{post}})$	$\mathrm{CV}(\widehat{t}_{\mathrm{post}})$
`clust87`	827 106 695	4.72e+15	8.31%
`REGION`	847 874 713	9.67e+15	11.60%

Table 3: Post-stratification 1 and 2 results

According to this one-sample drawing, post-stratifying using `clust87` is more accurate as the coefficient of variation is lower than post-stratifying using `REGION`.

2.4 Ratio estimator

The ratio estimator has the advantage to use an auxiliary variable (x) in order to estimate the total t_y. Indeed, the ratio estimator is said to be calibrated[2] on the total $t_x = \sum_{i \in U} x_i$.

The ratio estimator is p-unbiased and is defined as followed:

$$\widehat{t}_{\mathrm{rat}} = \frac{\widehat{t}_y}{\widehat{t}_x} t_x$$

[2] (*cf.* Chapter 6)

The variance formula of the ratio estimator is:

$$\widehat{V}(\widehat{t}_{\text{rat}}) = N^2(1 - \frac{n}{N})\frac{s_e^2}{n}$$

Where s_e^2 is the sample variance over the $e_i = y_i - \widehat{R}x_i$ ($\widehat{R} = \frac{\widehat{t}_y}{\widehat{t}_x}$) that we get using the first order of the Taylor expansion[3]:

$$s_e^2 = \frac{1}{n-1}\sum_{i \in s} e_i^2 = \frac{1}{n-1}\sum_{i \in s}(y_i - \widehat{R}x_i)^2$$

After calculations we get:

- $\widehat{t}_{\text{rat}} = 918\,014\,440$
- $\widehat{V}(\widehat{t}_{\text{rat}}) = 1.59e + 14$
- $\text{CV}(\widehat{t}_{\text{rat}}) = 1.37\%$

2.5 Regression estimator

Here is the definition of the regression estimator $\widehat{t}_{\text{reg}}$:

$$\widehat{t}_{\text{reg}} = N \times [\hat{\bar{t}}_y + \hat{\beta}(\bar{t}_x - \hat{\bar{t}}_x)]$$

Where:

- $\widehat{\beta} = \dfrac{\sum_{i \in s} d_i(x_i - \hat{\bar{t}}_x)(y_i - \hat{\bar{t}}_y)}{\sum_{i \in s} d_i(x_i - \widehat{\bar{t}}_x)^2}$
- $d_i = \pi_i^{-1} = \dfrac{N}{n}$

The formula of the variance of the regression estimator is defined as followed:

$$\widehat{V}(\widehat{t}_{\text{reg}}) = N^2(1 - \frac{n}{N})\frac{s_e^2}{n}$$

Where s_e^2 is the sample variance over the $e_i = y_i - x_i\hat{\beta}$.

After computations we get:

- $\widehat{t}_{\text{reg}} = 899\,950\,857$
- $\widehat{V}(\widehat{t}_{\text{reg}}) = 1.59e + 14$
- $\text{CV}(\widehat{t}_{\text{reg}}) = 1.39\%$

2.6 Calibrated estimator

In our dataset, auxiliary variable information is known for each unit in the population, which is the *complete auxiliary information* case. We define three auxiliary variables: `clust87`, `ACRES87` and `ACRES82`. As the potential gain in precision of calibrated estimator depends on correlation between the calibration variables and variable of interest, we firstly check the correlation matrix of quantitative variables.

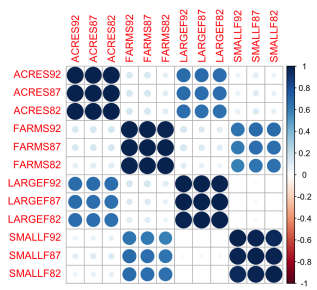

Figure 2: Correlation plot of the quantitative variables

[3] *cf.* Chapter 5

In particular, the correlation between y and `ACRES87` equals: $\rho_{xy} = 99.6\%$ and the correlation between y and `ACRES82` equals: $\rho_{x'y} = 99.32\%$

The population margin matrix is defined as:

Variable	Modality	Margin1	Margin2	Margin3	Margin4
`clust87`	4	762	740	747	743
`ACRES87`	0	941 765 666	0	0	0
`ACRES82`	0	963 376 742	0	0	0

Table 4: Margin matrix

Thus, we can apply calibration to the four survey designs we have defined previously. For these designs, the main difference lies in the initial weight:

- Under SRSWOR design, initial weight $d = \frac{N}{n}$ is identical for each unit
- Under stratification design, initial weight $d_i = \frac{N_h}{n_h}, i \in U_h$

2.6.1 Calibration method

We use linear method to calibrate. The reasons we use this method are:

- Raking method should be used in the context of calibration to known marginals of a frequency table of dimension 2 or higher. Here, we only have one categorical auxiliary variable, which is `clust87`
- For one sample, logit and truncated linear method work well and give similar results to linear method. The calibrated weights are alike among the three methods. Neverthelss, when it comes to simulation, logit and truncated linear methods don't exist a solution. The results won't converge within 2500 iterations

2.6.2 Calibration result

We get the following results for the calibrated estimator:

Survey design	$\widehat{t}_{\text{cal}}$	$\widehat{V}(\widehat{t}_{\text{cal}})$	$\text{CV}(\widehat{t}_{\text{cal}})$
SRSWOR	939 099 064	1.71e+11	0.0441%
ST1	925 502 866	1.65e+11	0.0439%
ST2	917 827 315	1.51e+11	0.0423%
ST3	917 003 922	2.15e+11	0.0506%

Table 5: Comparison of calibration estimators based on different survey designs

We can tell from the table that calibrated estimator is efficient, as it has a small coefficient of variation. Also, we observe that calibrated estimators for the four survey designs are very similar, although the estimates of these designs are divergent. The calibration imposes constraints on the weights to ensure that the weighted sample still satisfy the known population totals. As a result, it helps us to obtain consistent estimates.

3 Simulations

In this section, we want to test the efficiency of the 4 different sampling designs we have defined and compare them together. As well as comparing the 5 different estimators and check for their unbiasedness.

To do so, we will draw for each sampling design or estimator, 1000 samples (hence 1000 simulations). Afterwards, applying Monte-Carlo properties, we will be able to compute more precisely the estimators, variances, design effects and so on. Even if we know that drawing all possible samples is out of reach.

3.1 Sampling designs comparison

Here are the histograms for the Horvitz-Thompson estimator of the total according to the sampling design:

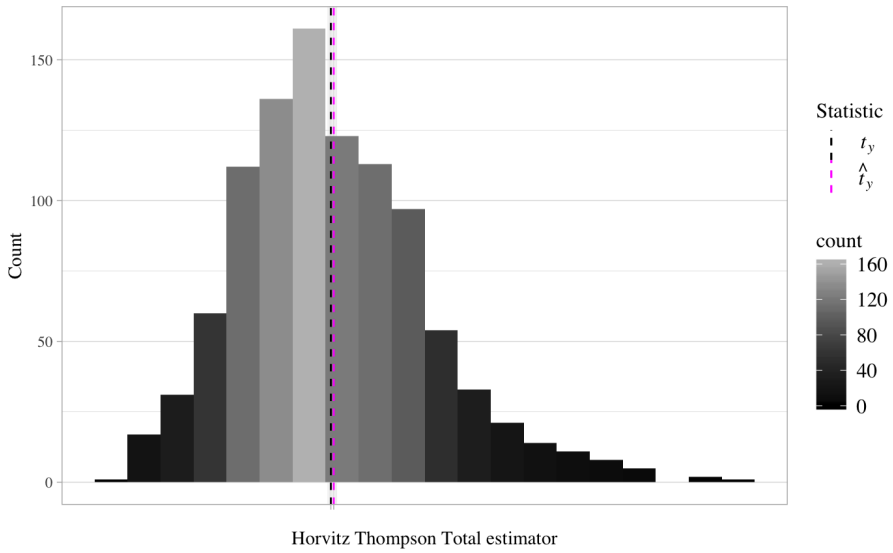

Figure 3: Monte Carlo simulations results for HT estimator applying SRSWOR design

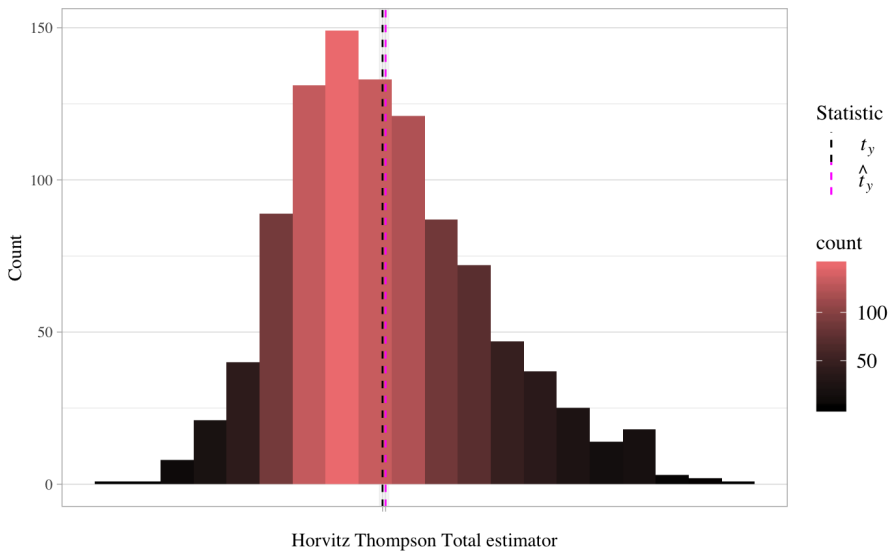

Figure 4: Monte Carlo simulations results for HT estimator applying STRSWOR design (allocation proportional to x)

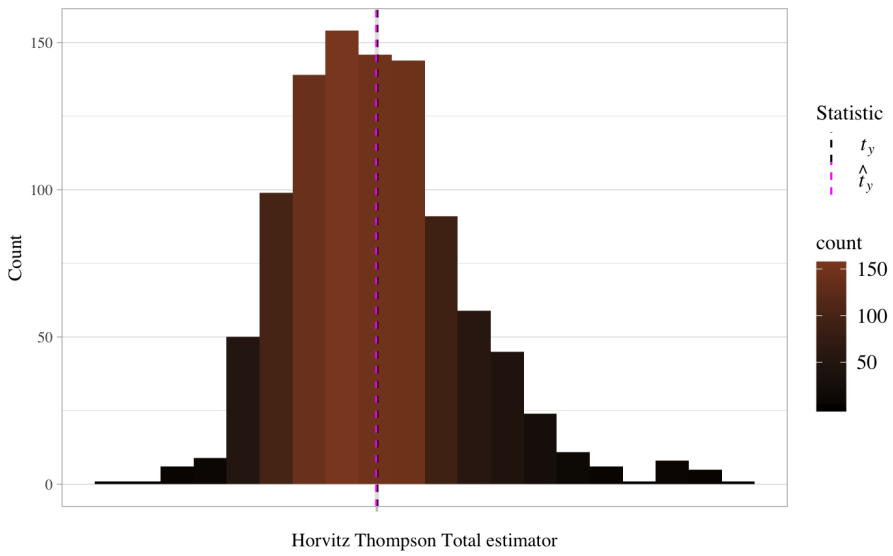

Figure 5: Monte Carlo simulations results for HT estimator applying STRSWOR design (Neyman allocation

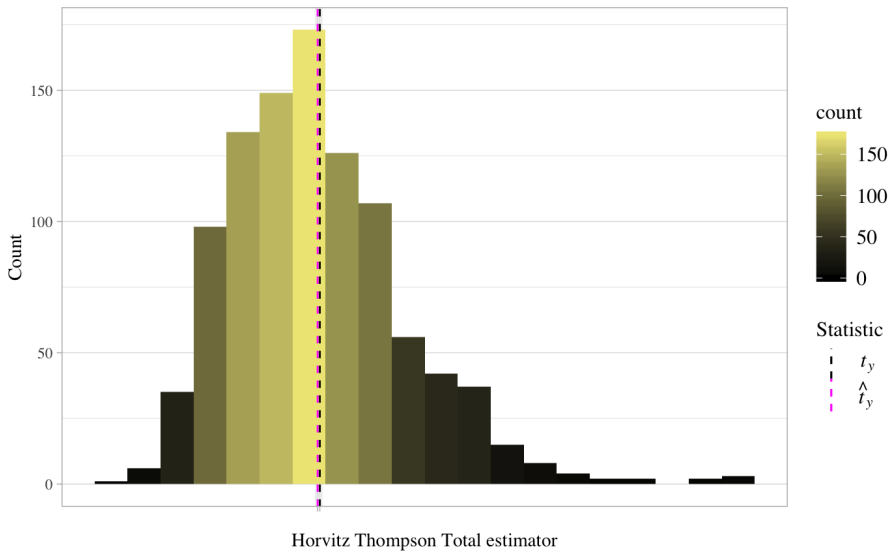

Figure 6: Monte Carlo simulations results for HT estimator applying STRSWOR design (proportional allocation

Sampling design	$\widehat{t}_{y,\mathrm{ST}}$	$\widehat{V}_{\mathrm{ST}}(\widehat{t}_{y,\mathrm{ST}})$	$\mathrm{CV}(\widehat{t}_{y,\mathrm{ST}})$	deff
SRSWOR	927 659 519	2.08e+16	15.55%	1
ST1	926 110 337	7.85e+15	9.56%	37.70%
ST2	922 011 226	8.48e+15	9.99%	40.76%
ST3	920 576 716	1.22e+16	11.99%	58.52%

Table 6: Sampling designs results by applying Monte-Carlo technique

Knowing that $t_y = 923\,355\,862$, sampling design ST2 is closer to the true total than the others. However sampling design ST1 performs better because this latter shows a lower variance, lower coefficient of variation and a lower design effect compared to the SRSWOR design. To conclude, one would recommend to use the auxiliary variable x in order to calculate the size of each sample stratum as it results an estimator close to the true total, a low coefficient of variation and a design effect lower than 1 (the lowest among all stratified designs performed).

In order to have an enhanced visual interpretation of these results, here are the plotted densities of the previous seen sampling designs.

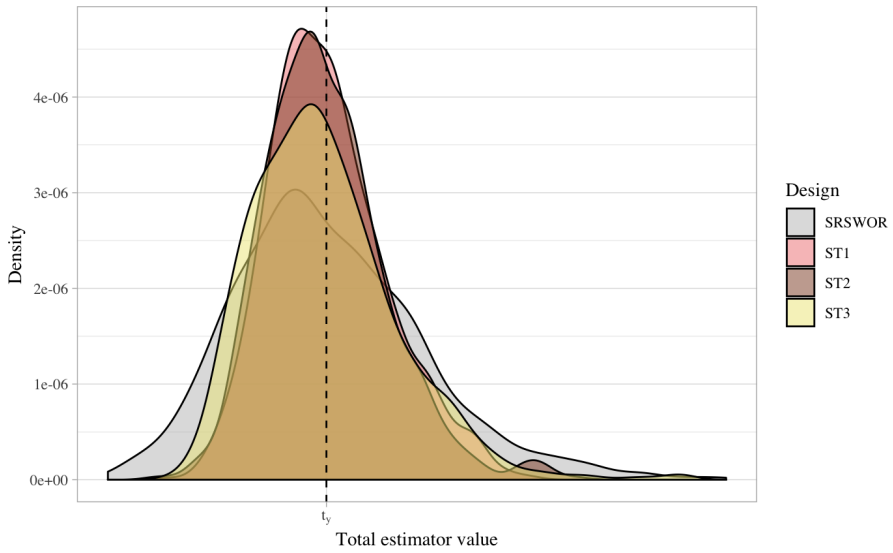

Figure 7: Sampling designs comparison applying Monte Carlo

The interpretation is that simple, the less spread the density, the lower the variance. The higher the density, the closer to the true total the estimator is applying the given sampling design.

In this case, we notice that ST1 and ST2 are both sampling designs providing improved results compared to the others. Of course, SRSWOR is the one which performs the worst.

3.2 Estimators comparison

Here are the histograms drawn from the 1000 simulations and applying the Monte-Carlo method to calculate each estimator:

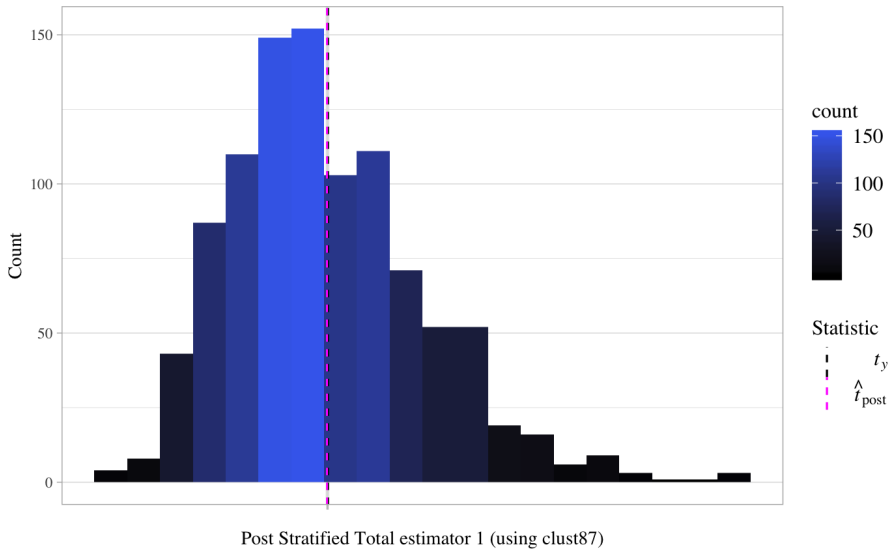

Figure 8: Monte Carlo simulations results for the post-stratified estimator 1 (`clust87`) applying SRSWOR design

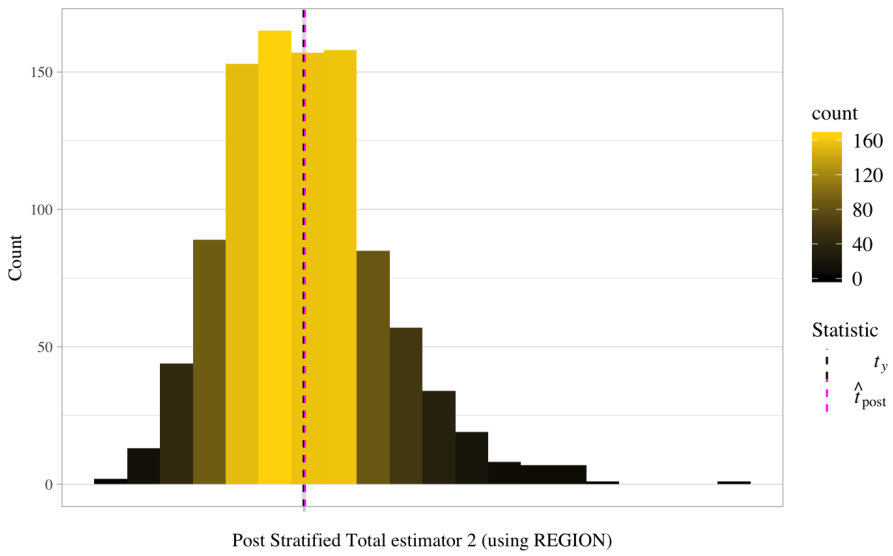

Figure 9: Monte Carlo simulations results for the post-stratified estimator 1 (`REGION`) applying SRSWOR design

Before going deeper into the comparison of further estimator, let us compare both post-stratified estimators:

Post-stratified estimator	$\widehat{t}_{\text{post}}$	$\widehat{V}(\widehat{t}_{\text{post}})$	$\text{CV}(\widehat{t}_{\text{post}})$	error
PS1	922 186 703	1.24e+16	12.10%	0.127%
PS2	925 720 770	1.78e+16	14.41%	0.256%

Table 7: Post-stratified estimators by applying Monte-Carlo technique

Based on these 1000 simulations, we can clearly conclude that post-stratifying according to the variable `clust87` (PS1) enhanced all results (estimator, variance, coefficient of variation, error[4]) compared to the post-stratification using `REGION` (PS2).

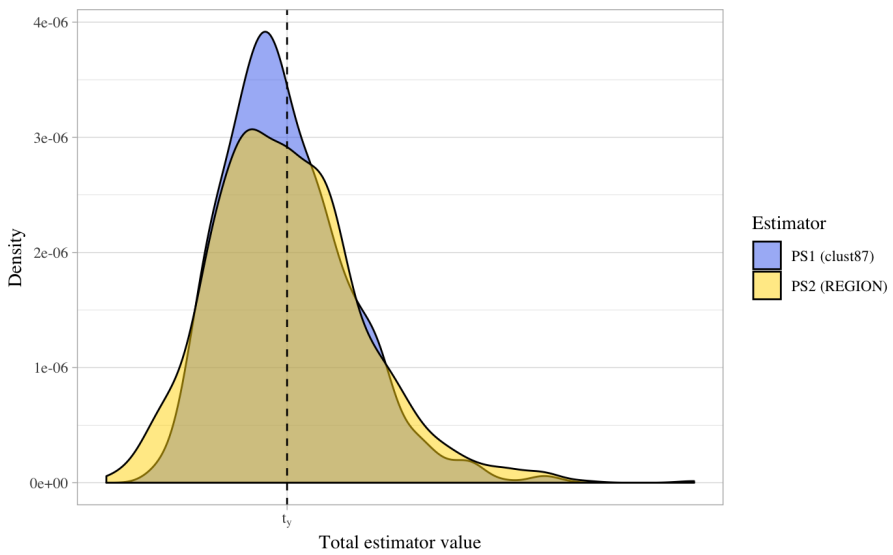

Figure 10: Monte-Carlo post-stratified estimators comparison applying SRSWOR design

[4] $\text{error} = \dfrac{\widehat{t}_{\text{post}} - t_y}{t_y} \times 100\%$

We can continue by checking the results for the ratio, regression and calibrated estimator:

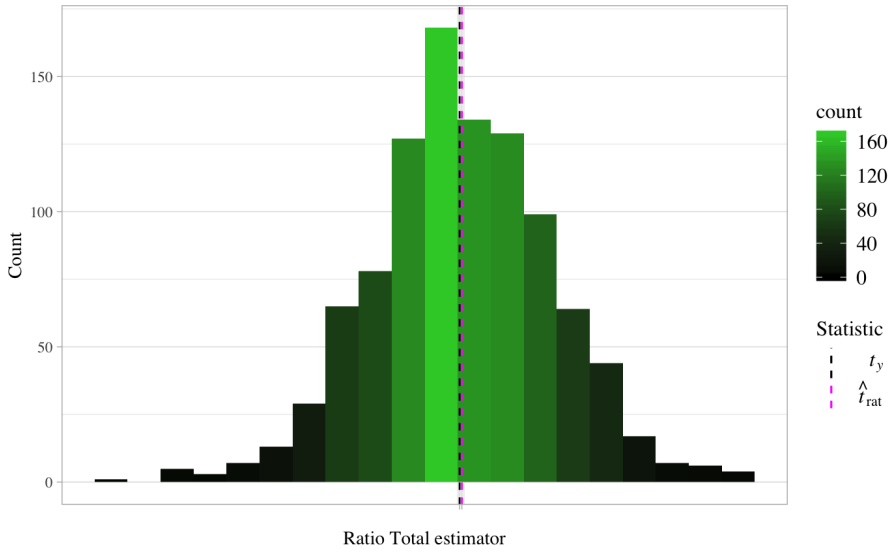

Figure 11: Monte Carlo simulations results for the ratio estimator applying SRSWOR design

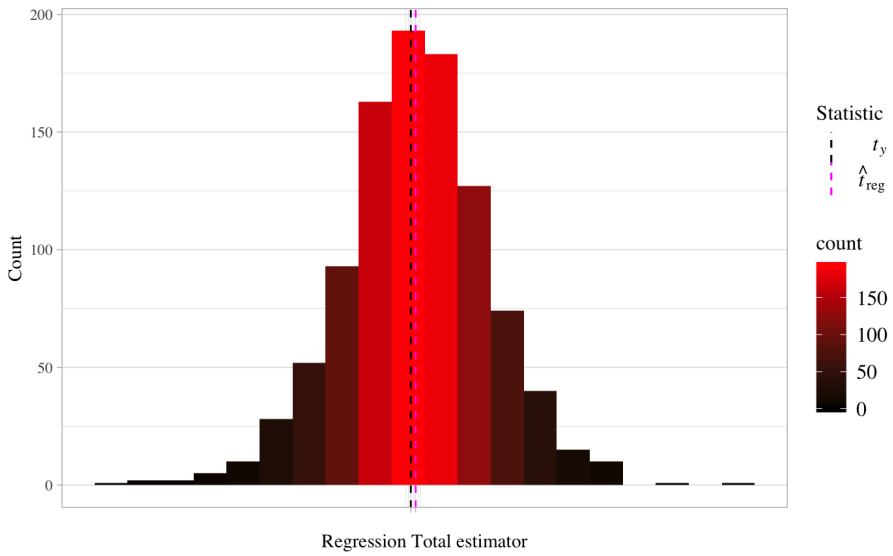

Figure 12: Monte Carlo simulations results for the regression estimator applying SRSWOR design

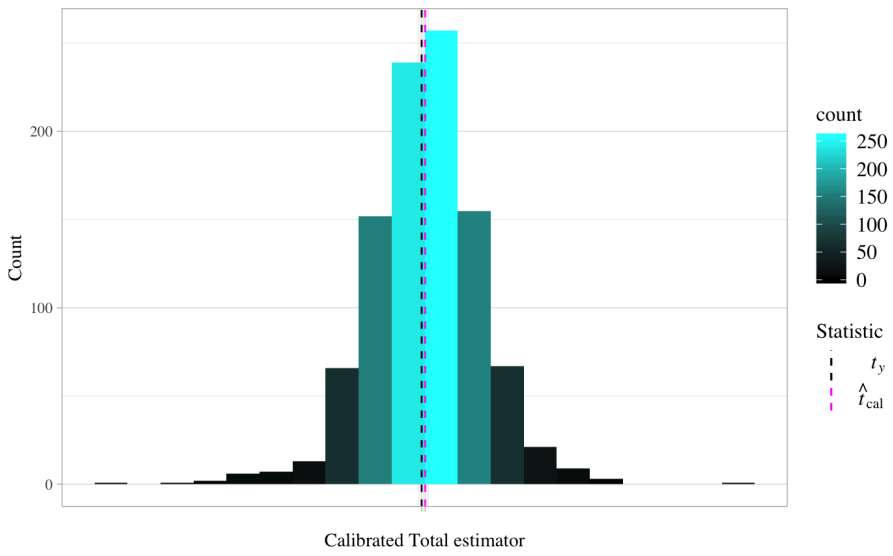

Figure 13: Monte Carlo simulations results for the calibrated estimator applying SRSWOR design

We get the following results over the 1000 simulations:

Estimator	$\widehat{t}_{\text{esti}}$	$\widehat{V}(\widehat{t}_{\text{esti}})$	$\text{CV}(\widehat{t}_{\text{esti}})$	error
Ratio	923 676 443	1.60e+14	1.37%	0.035%
Regression	924 218 484	1.73e+14	1.42%	0.093%
Calibrated	924 237 232	2.05e+14	1.55%	0.095%

Table 8: Ratio, regression and calibrated estimators using Monte-Carlo technique

Based on these results, the ratio estimator is the best one as it has the lowest variance and coefficient of variation. Nonetheless they are all very performant.

It can be explained due to the fact that, the ratio does not need any assumption regarding the variance of residuals whereas the regression and calibrated estimator do. Furthermore the ratio estimator performs well when there is no intercept (or intercept = 0) which is the case here (*cf.* 1.3). Here are the plotted densities for those three estimators:

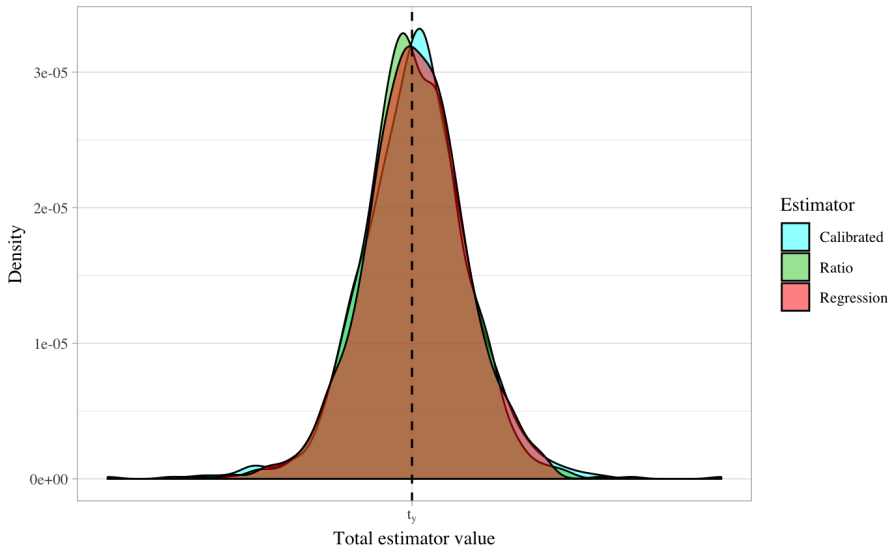

Figure 14: Monte-Carlo ratio, regression and calibrated estimators comparison applying SRSWOR design

Now, it can be relevant to compare one of those three estimators with the post-stratified estimators. Still computed with the help of the Monte-Carlo technique.

Numerically, taking the example of the ratio estimator we have:

$$\widehat{V}(\widehat{t}_{\text{rat}}) << \widehat{V}(\widehat{t}_{\text{post1}})$$

$$\Leftrightarrow$$

$$1.60e + 14 << 1.24e + 16$$

Similar results can be showed for $\text{CV}(\widehat{t}_{\text{rat}})$ and $\text{error}_{\text{rat}}$.

Here is the final table displaying all estimators and the plotted densities for both post-stratified estimators and the ratio estimator which is way more accurate the post-stratified ones:

Estimator	$\widehat{t}_{\text{esti}}$	$\widehat{V}(\widehat{t}_{\text{esti}})$	$\text{CV}(\widehat{t}_{\text{esti}})$	error
PS1	922 186 703	1.24e+16	12.10%	0.127%
PS2	925 720 770	1.78e+16	14.41%	0.256%
Ratio	923 676 443	1.60e+14	1.37%	0.035%
Regression	924 218 484	1.73e+14	1.42%	0.093%
Calibrated	924 237 232	2.05e+14	1.55%	0.095%

Table 9: All estimators using Monte-Carlo technique

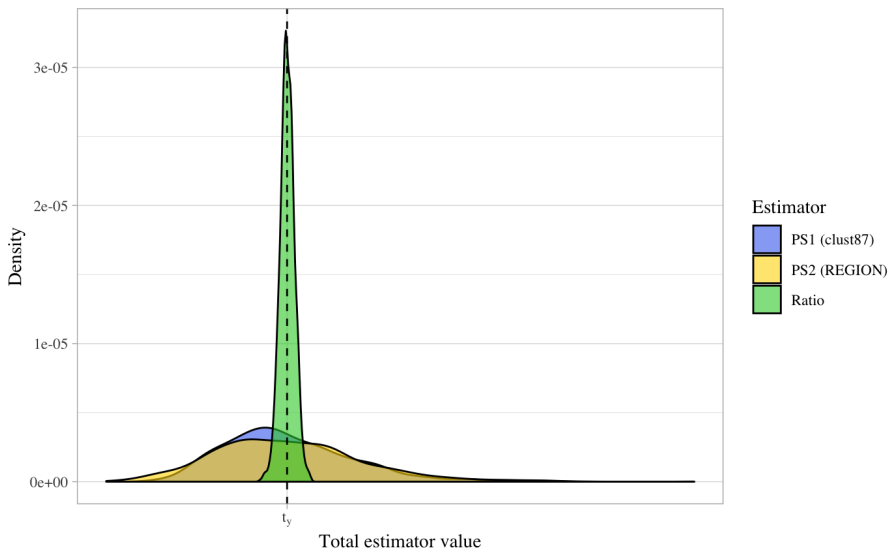

Figure 15: Monte-Carlo estimators comparison between post-stratified and ratio estimators applying SRSWOR

On this plot we clearly notice that the ratio has a lower variance and is much more accurate than the post-stratified estimators. And this is logical since it takes into account the auxiliary variable x which is almost perfectly linearly correlated with y.

4 Appendix

This is an Rmarkdown file. Do not forget to insert your own path file for the file `farms.csv` in the script (line 54).

```
1  ---
2  title: "Farms"
3  author: "Bruno Pilarczyk & Shuyao Li"
4  date: "11/8/2019"
5  output: html_document
6  ---
7
8  ```{r setup, include=FALSE}
9  knitr::opts_chunk$set(echo = TRUE)
10 ```
11
12 ```{r setup, include=FALSE}
13 knitr::opts_chunk$set(echo = TRUE)
14 rm(list=ls())
15 graphics.off()
16 ```
17
18 ```{r packages, include = FALSE}
19 if (require(survey) == FALSE)
20   install.packages("survey")
21 library("survey")
22
23 if (require(sampling) == FALSE)
24   install.packages("sampling")
25 library("sampling")
26
27 if (require(ggplot2) == FALSE)
28   install.packages("ggplot2")
29 library("ggplot2")
30
31 if (require(icarus) == FALSE)
32   install.packages("icarus")
33 library("icarus")
34
35 if (require(RColorBrewer) == FALSE)
36   install.packages("RColorBrewer")
37 library("RColorBrewer")
38
39 if (require(Hmisc) == FALSE)
40   install.packages("Hmisc")
41 library("Hmisc")
42
43 if (require(latex2exp) == FALSE)
44   install.packages("latex2exp")
45 library("latex2exp")
46
47 if (require(corrplot) == FALSE)
48   install.packages("corrplot")
49 library("corrplot")
50 ```
51
52 ```{r data importation, include = FALSE}
53 #pathname of the rec99htegne.csv file
54 my_path = "/Users/brunopilarczyk/Documents/Cours/M2 Stat Eco/S1/Survey Sampling/Farms/farms.csv
       "
55 farms = read.csv(file = my_path, header = T, dec = ".", sep = ",")
56 N = nrow(farms)
57 n = 80 #sample size
58 farms$weights = rep(N/n, N)
59 attach(farms) # allows to handle variables easily
60 ```
61
62 #1 Preliminary analysis
63 ##1.1
64 ```{r init}
65 head(farms)
66 str(farms)
67
68 farms$CODE = paste0(farms$COUNTY,farms$STATE, sep= " ")
69 #there are duplicated county names, we have to use COUNTY and STATE together to identify a
       place
70 sum(ifelse(duplicated(farms$CODE),1,0)) == 0 #no duplicates
71 farms = data.frame(farms)
72 ```
73
74 ```{r variables for stratification and post-stratification}
75 scatter1 = ggplot(farms, aes(x = LARGEF87, y = ACRES92)) +
76   geom_point() +
77   ggtitle("Scatterplot of ACRES92 on LARGEF87") +
78   theme_light(base_family = "Times")
```

```
scatter1

# LARGEF87 coulbe a relevant variables once transformed into a categorical variable

summary(LARGEF87)

# Let's define clust_L87 as a categorical variable:
# clust87 = 1 if LARGEF87 =< 8
# clust87 = 2 if  8 < LARGEF87  =< 27
# clust87 = 3 if  27 < LARGEF87 =< 69
# clust87 = 4 if LARGEF87 > 69

quartiles = quantile(farms$LARGEF87, seq(0, 1, .25))
farms$clust87 = cut(farms$LARGEF87, quartiles, include.lowest = TRUE)
farms$clust87 = factor(farms$clust87, labels = c(1,2,3,4))

table(farms$clust87)
attach(farms)

# the other variable for post-stratification will be REGION
```


#1.2
```{r 2.2}
T_y = sum(ACRES92); T_y
V_y = var(ACRES92); V_y
```

#1.3
```{r 2.3}
scatter2 = ggplot(farms, aes(x = ACRES87, y = ACRES92)) +
  geom_point() +
  theme_light(base_family = "Times")
scatter2
```

#2 A single sample

##2.1 SRSWOR design
```{r 3.1}
set.seed(7)

si.farms = srswor(n,N)
ech.si = svydesign( id=~CODE, weights = rep(N/n, n),
  fpc = rep(N,n),
  data = farms[which(si.farms==1),])

est = svytotal(~ACRES92,ech.si)
Y_HT_SR = est[1]; Y_HT_SR
V_HT_SR = SE(est)^2 ;V_HT_SR
CV_HT_SR = SE(est) / Y_HT_SR; CV_HT_SR
```
##2.2 Stratified designs 1 and 2

###2.2.1 Stratified design 1: Allocation proportional to ACRES87
```{r sample in each clust87}
# We will use the allocation proportional to ACRES87 (auxiliary variable) and then look at each stratum of clust87
set.seed(7)

N_clust87 = table(clust87)

ACRES87_sum = tapply(ACRES87, clust87, sum)
ech.prop87 = round(n*ACRES87_sum/sum(ACRES87_sum))
ech.prop87[4] = ech.prop87[4] - 1
ech.prop87

prop87.rec = strata(farms,
                    "clust87",
                    size=c(ech.prop87[1],ech.prop87[2],ech.prop87[3],ech.prop87[4]),
                    method="srswor")

prop87.data = getdata(farms,prop87.rec)
```


```{r stratifying proportionally to an auxiliary variable}
weight_prop87 = c(rep(N_clust87[1]/ech.prop87[1], ech.prop87[1]),
                  rep(N_clust87[2]/ech.prop87[2], ech.prop87[2]),
                  rep(N_clust87[3]/ech.prop87[3], ech.prop87[3]),
                  rep(N_clust87[4]/ech.prop87[4], ech.prop87[4]))

sample.prop87 = svydesign(id=~CODE, strata = ~clust87, weights = weight_prop87,
                     fpc = c(rep(ech.prop87[1]/N_clust87[1], ech.prop87[1]),
                             rep(ech.prop87[2]/N_clust87[2], ech.prop87[2]),
                             rep(ech.prop87[3]/N_clust87[3], ech.prop87[3]),
```

```
                                rep(ech.prop87[4]/N_clust87[4], ech.prop87[4])),
                        data = prop87.data)

res_prop87 = svytotal(~ACRES92,sample.prop87)

Y_prop87 = res_prop87[1];Y_prop87
SD_prop87 = SE(res_prop87);SD_prop87
V_prop87 = SD_prop87^2;V_prop87
CV_prop87 = SD_prop87/Y_prop87;CV_prop87
deff_prop87 = V_prop87/V_HT_SR;deff_prop87
```

2.2.2 Stratified design 2: Neyman allocation

```{r defining the sample strata}
set.seed(7)
sd_87 = tapply(ACRES92, clust87, sd)

N_sd_87 = numeric(4)
for (i in 1:4){
  N_sd_87[i] = N_clust87[i] * sd_87[i]
}
sum_N_sd_87 = sum(N_sd_87)

n_opt = numeric(4)
for (i in 1:4){
  n_opt[i] = round(n*(N_clust87[i]*sd_87[i])/(sum_N_sd_87))
}
n_opt

neyman.rec = strata(farms,"clust87", size=c(n_opt[1],n_opt[2],n_opt[3],n_opt[4]), method="srswor")
neyman.data = getdata(farms,neyman.rec)
```

```{r stratifying with the neyman allocation}
weight_neyman = c(rep(N_clust87[1]/n_opt[1], n_opt[1]),
                  rep(N_clust87[2]/n_opt[2], n_opt[2]),
                  rep(N_clust87[3]/n_opt[3], n_opt[3]),
                  rep(N_clust87[4]/n_opt[4], n_opt[4]))

sample.neyman = svydesign(id=~CODE, strata = ~clust87, weights = weight_neyman,
                        fpc = c(rep(n_opt[1]/N_clust87[1], n_opt[1]),
                                rep(n_opt[2]/N_clust87[2], n_opt[2]),
                                rep(n_opt[3]/N_clust87[3], n_opt[3]),
                                rep(n_opt[4]/N_clust87[4], n_opt[4])),
                        data = neyman.data)

res_neyman = svytotal(~ACRES92,sample.neyman)

Y_neyman = res_neyman[1];Y_neyman
SD_neyman = SE(res_neyman);SD_neyman
V_neyman = SD_neyman^2;V_neyman
CV_neyman = SD_neyman/Y_neyman;CV_neyman
deff_neyman = V_neyman/V_HT_SR;deff_neyman
```

2.2.3 Stratified design 3: Proportional allocation

```{r defining the sample strat}
n_prop = round((n/N)*N_clust87)
sum(n_prop) == n
n_prop

prop.rec = strata(farms,"clust87",
                        size=c(n_prop[1],n_prop[2],n_prop[3],n_prop[4]),
                        method="srswor")

prop.data = getdata(farms,prop.rec)
```

```{r stratifying with the proportional allocation}
weight_prop = c(rep(N_clust87[1]/n_prop[1], n_prop[1]),
                  rep(N_clust87[2]/n_prop[2], n_prop[2]),
                  rep(N_clust87[3]/n_prop[3], n_prop[3]),
                  rep(N_clust87[4]/n_prop[4], n_prop[4]))

sample.prop = svydesign(id=~CODE, strata = ~clust87, weights = weight_prop,
                        fpc = c(rep(n_prop[1]/N_clust87[1], n_prop[1]),
                                rep(n_prop[2]/N_clust87[2], n_prop[2]),
                                rep(n_prop[3]/N_clust87[3], n_prop[3]),
                                rep(n_prop[4]/N_clust87[4], n_prop[4])),
                        data = prop.data)

res_prop = svytotal(~ACRES92,sample.prop)
res_prop

Y_prop = res_prop[1];Y_prop
```

```
SD_prop = SE(res_prop);SD_prop
V_prop = SD_prop^2;V_prop
CV_prop = SD_prop/Y_prop;CV_prop
deff_prop = V_prop/V_HT_SR;deff_prop
```

```
##2.3 Post-Stratification 1 and 2 : clust87 and REGION
###Variable 1: clust87
```{r 2.3 var2}
set.seed(7)
N_clust87 = table(clust87)
ech.post_clust87 = postStratify(ech.si, ~clust87, N_clust87, partial = T)
est.post_clust87 = svytotal(~ACRES92, ech.post_clust87)

Y_POST_clust87 = est.post_clust87[1];Y_POST_clust87
V_POST_clust87 = SE(est.post_clust87)^2 ;V_POST_clust87
CV_POST_clust87 = SE(est.post_clust87) / Y_POST_clust87; CV_POST_clust87
```

```
###Variable 2: REGION
```{r 2.3 var2}
set.seed(7)
N_REGION = table(REGION)
ech.post_REGION = postStratify(ech.si, ~REGION, N_REGION, partial = T)
est.post_REGION = svytotal(~ACRES92, ech.post_REGION)

Y_POST_REGION = est.post_REGION[1];Y_POST_REGION
V_POST_REGION = SE(est.post_REGION)^2 ;V_POST_REGION
CV_POST_REGION = SE(est.post_REGION) / Y_POST_REGION; CV_POST_REGION
```

```
##2.4 Ratio estimator
```{r 2.4 Ratio Estimator}
set.seed(7)
est.ratio = svyratio(~ACRES92, ~ACRES87, ech.si)
res.ratio = predict(est.ratio, total = sum(ACRES87))
Y_ratio = res.ratio$total;Y_ratio
SD_ratio = res.ratio$se;SD_ratio
V_ratio = SD_ratio^2;V_ratio
CV_ratio = SD_ratio/Y_ratio;CV_ratio
```

```
##2.5 Regression estimator
```{r 2.5 Regression Estimator}
set.seed(7)
ech.si.reg = calibrate(ech.si, ~ACRES87, c(N, sum(ACRES92)))
res.si.reg = svytotal(~ACRES92, ech.si.reg)
Y_reg = res.si.reg[1];Y_reg
SD_reg = SE(res.si.reg);SD_reg
V_reg = SD_reg^2;V_reg
CV_reg = SD_reg/Y_reg;CV_reg
```


##2.6 Calibration

```{r correlation of ACRES87 and ACRES82 with ACRES92}
df = farms[c("ACRES92", "ACRES87", "ACRES82",
             "FARMS92", "FARMS87", "FARMS82",
             "LARGEF92", "LARGEF87", "LARGEF82",
             "SMALLF92","SMALLF87","SMALLF82")]


cor(ACRES87, ACRES92)
cor(ACRES82, ACRES92)

corrplot(cor(df))
```

```{r 2.6 margin matrix}

mar1 = c("clust87",4,762,740,747,743)
mar2 = c("ACRES87", 0, sum(farms$ACRES87),0,0,0)
mar3 = c("ACRES82", 0, sum(farms$ACRES82),0,0,0)

# we don't use state because there are some chances that the sample didn't pick every state
marginmatrix = rbind(mar1, mar2,mar3)
marginmatrix

```

```{r 2.6 calibration on SRSWOR design}
sample = farms[which(si.farms==1),]
calib1 <- calibration(data = sample ,
                      marginMatrix = marginmatrix,
                      colWeights = "weights",
                      method = "linear",
```

```
                      description = T)

Y_cal1 = weightedTotal(sample$ACRES92, calib1); Y_cal1
SD_cal1 = sqrt(wtd.var(sample$ACRES92, calib1)); SD_cal1
V_cal1 = SD_cal1^2;V_cal1
CV_cal1= SD_cal1/Y_cal1;CV_cal1
```


```{r 2.6 calibration on ST1 design }
prop87.data$weights = weight_prop87
calib2 <- calibration(data = prop87.data,
                      marginMatrix = marginmatrix,
                      colWeights = "weights",
                      method = "linear",
                      description = F)

Y_cal2 = weightedTotal(prop87.data$ACRES92, calib2); Y_cal2
SD_cal2 = sqrt(wtd.var(prop87.data$ACRES92, calib2)); SD_cal2
V_cal2 = SD_cal2^2;V_cal2
CV_cal2= SD_cal2/Y_cal2;CV_cal2
```

```{r 2.6 calibration on ST2 design }
neyman.data$weights = weight_neyman
calib3 <- calibration(data = neyman.data,
                      marginMatrix = marginmatrix,
                      colWeights = "weights",
                      method = "linear",
                      description = F)

Y_cal3 = weightedTotal(neyman.data$ACRES92, calib3); Y_cal3
SD_cal3 = sqrt(wtd.var(neyman.data$ACRES92, calib3)); SD_cal3
V_cal3 = SD_cal3^2;V_cal3
CV_cal3= SD_cal3/Y_cal3;CV_cal3
```

```{r 2.6 calibration on ST3 design }
prop.data$weights = weight_prop
calib4 <- calibration(data = prop.data,
                      marginMatrix = marginmatrix,
                      colWeights = "weights",
                      method = "linear",
                      description = F)

Y_cal4 = weightedTotal(prop.data$ACRES92, calib4); Y_cal4
SD_cal4 = sqrt(wtd.var(prop.data$ACRES92, calib4)); SD_cal4
V_cal4 = SD_cal4^2;V_cal4
CV_cal4= SD_cal4/Y_cal4;CV_cal4
```


#3 Simulations

#3.1 SRSWOR design

```{r 2.4.1 (6), echo = FALSE}
# Initialization
set.seed(7)
nsimu = 1000
nb.simul = matrix(1:nsimu, nsimu, 1)
tu.esti_HT = matrix(1, nsimu, 1)
var.esti.tu_HT = matrix(1, nsimu, 1)

#Simulations
for (i in 1:nsimu)
{
  srswor.farms = srswor(n, N)
  design.srswor = svydesign(id=~CODE,
                      weights = rep(N/n, n),
                      fpc = rep(N,n),
                      data = farms[which(srswor.farms==1),])
 a = svytotal(~ACRES92, design.srswor)
 tu.esti_HT[i] <- a[1]
 var.esti.tu_HT[i] <- SE(a)^2
}

#Monte Carlo: mean and standard deviation
Y_MC_HT = mean(tu.esti_HT);Y_MC_HT
V_MC_HT = var(tu.esti_HT);V_MC_HT
SD_MC_HT = sqrt(var(tu.esti_HT));SD_MC_HT
CV_MC_HT = SD_MC_HT/Y_MC_HT;CV_MC_HT

#Histogram
HT = data.frame(values = tu.esti_HT)
qplot(x = HT$values, fill= ..count.., geom="histogram", bins = 20) +
  geom_vline(aes(xintercept = Y_MC_HT,
```

```
            color = "true"),
            size = 0.5,
            linetype = "dashed",
            show.legend = T) +
  geom_vline(aes(xintercept = T_y,
            color = "estimate"),
            size = 0.5,
            linetype = "dashed",
            show.legend = T) +
  xlab("Horvitz Thompson Total estimator") +
  ylab("Count") +
  scale_fill_gradient(low="black", high="grey") +
  scale_x_continuous(breaks=c(T_y, Y_MC_HT) ,labels=c(" "," ")) +
  scale_color_manual(name = "Statistic",
                     labels=c(expression(italic(t)[italic(y)]),
                              expression(hat(italic(t))[italic(y)])),
                     values=c("black", "magenta")) +
  theme_light(base_family = "Times") +
  theme(legend.text = element_text(size = 12),
        legend.title = element_text(size = 12))
```

#3.2 Stratified designs 1 and 2

```{r Stratified design 1: allocation proportional to ACRES87}
# Initialization
set.seed(7)
nsimu = 1000
nb.simul = matrix(1:nsimu, nsimu, 1)
tu.esti_st = matrix(1, nsimu, 1)
var.esti.tu_st = matrix(1, nsimu, 1)


for (i in 1:nsimu){
  prop87.rec = strata(farms,
                    "clust87",
                    size=c(ech.prop87[1],ech.prop87[2],ech.prop87[3],ech.prop87[4]),
                    method="srswor")

  prop87.data = getdata(farms,prop87.rec)

  weight_prop87 = c(rep(N_clust87[1]/ech.prop87[1], ech.prop87[1]),
                    rep(N_clust87[2]/ech.prop87[2], ech.prop87[2]),
                    rep(N_clust87[3]/ech.prop87[3], ech.prop87[3]),
                    rep(N_clust87[4]/ech.prop87[4], ech.prop87[4]))

  sample.prop87 = svydesign(id=~CODE, strata = ~clust87, weights = weight_prop87,
                        fpc = c(rep(ech.prop87[1]/N_clust87[1], ech.prop87[1]),
                                rep(ech.prop87[2]/N_clust87[2], ech.prop87[2]),
                                rep(ech.prop87[3]/N_clust87[3], ech.prop87[3]),
                                rep(ech.prop87[4]/N_clust87[4], ech.prop87[4])),
                        data = prop87.data)

  res_prop87 = svytotal(~ACRES92,sample.prop87)
  tu.esti_st[i] <- res_prop87[1]
  var.esti.tu_st[i] <- SE(res_prop87)^2
}

#Monte Carlo: mean and standard deviation
Y_MC_st1 = mean(tu.esti_st);Y_MC_st1
V_MC_st1 = var(tu.esti_st);V_MC_st1
SD_MC_st1 = sqrt(var(tu.esti_st));SD_MC_st1
CV_MC_st1 = SD_MC_st1/Y_MC_st1;CV_MC_st1

#Histogram
st1 = data.frame(values = tu.esti_st)
qplot(x = st1$values, fill= ..count.., geom="histogram", bins = 20) +
  geom_vline(aes(xintercept = T_y, color = "black"), size = 0.5, linetype = "dashed") +
  geom_vline(aes(xintercept = Y_MC_st1, color = "purple"), size = 0.5, linetype = "dashed") +
  xlab("Horvitz Thompson Total estimator") +
  ylab("Count") +
  scale_fill_gradient(low="black", high="lightcoral") +
  scale_x_continuous(breaks=c(T_y, Y_MC_st1) ,labels=c(" "," ")) +
  scale_color_manual(name = "Statistic",
                     labels=c(expression(italic(t)[italic(y)]),
                              expression(hat(italic(t))[italic(y)])),
                     values=c("black", "magenta")) +
  theme_light(base_family = "Times") +
  theme(legend.text = element_text(size = 12),
        legend.title = element_text(size = 12))
```

```{r Stratified design 2: Neyman allocation}
# Initialization
set.seed(7)
nsimu = 1000
nb.simul = matrix(1:nsimu, nsimu, 1)
```

```
tu.esti_st = matrix(1, nsimu, 1)
var.esti.tu_st = matrix(1, nsimu, 1)


for (i in 1:nsimu){
  neyman.rec = strata(farms,
                      "clust87",
                      size=c(n_opt[1],n_opt[2],n_opt[3],n_opt[4]),
                      method="srswor")

  neyman.data = getdata(farms,neyman.rec)

  weight_neyman = c(rep(N_clust87[1]/n_opt[1], n_opt[1]),
                      rep(N_clust87[2]/n_opt[2], n_opt[2]),
                      rep(N_clust87[3]/n_opt[3], n_opt[3]),
                      rep(N_clust87[4]/n_opt[4], n_opt[4]))

  sample.neyman = svydesign(id=~CODE, strata = ~clust87, weights = weight_neyman,
                          fpc = c(rep(n_opt[1]/N_clust87[1], n_opt[1]),
                                  rep(n_opt[2]/N_clust87[2], n_opt[2]),
                                  rep(n_opt[3]/N_clust87[3], n_opt[3]),
                                  rep(n_opt[4]/N_clust87[4], n_opt[4])),
                          data = neyman.data)

  res_neyman = svytotal(~ACRES92,sample.neyman)
  tu.esti_st[i] <- res_neyman[1]
  var.esti.tu_st[i] <- SE(res_neyman)^2
}

#Monte Carlo: mean and standard deviation
Y_MC_st2 = mean(tu.esti_st);Y_MC_st2
V_MC_st2 = var(tu.esti_st);V_MC_st2
SD_MC_st2 = sqrt(var(tu.esti_st));SD_MC_st2
CV_MC_st2 = SD_MC_st1/Y_MC_st1;CV_MC_st2

#Histogram
st2 = data.frame(values = tu.esti_st)
qplot(x = st2$values, fill= ..count.., geom = "histogram", bins = 20) +
  geom_vline(aes(xintercept = T_y, color = "black"), size = 0.5, linetype = "dashed") +
  geom_vline(aes(xintercept = Y_MC_st2, color = "purple"), size = 0.5, linetype = "dashed") +
  xlab("Horvitz Thompson Total estimator") +
  ylab("Count") +
  scale_fill_gradient(low = "black", high = "sienna4") +
  scale_x_continuous(breaks=c(T_y, Y_MC_st2) ,labels=c(" "," ")) +
  scale_color_manual(name = "Statistic",
                     labels=c(expression(italic(t)[italic(y)]),
                              expression(hat(italic(t))[italic(y)])),
                     values=c("black", "magenta")) +
  theme_light(base_family = "Times") +
  theme(legend.text = element_text(size = 12),
        legend.title = element_text(size = 12))
```

```{r Stratified design 3: Proportional allocation}
# Initialization
set.seed(7)
nsimu = 1000
nb.simul = matrix(1:nsimu, nsimu, 1)
tu.esti_st = matrix(1, nsimu, 1)
var.esti.tu_st = matrix(1, nsimu, 1)


for (i in 1:nsimu){
  prop.rec = strata(farms,
                      "clust87",
                      size=c(n_prop[1],n_prop[2],n_prop[3],n_prop[4]),
                      method="srswor")

  prop.data = getdata(farms,prop.rec)

  weight_prop = c(rep(N_clust87[1]/n_prop[1], n_prop[1]),
                      rep(N_clust87[2]/n_prop[2], n_prop[2]),
                      rep(N_clust87[3]/n_prop[3], n_prop[3]),
                      rep(N_clust87[4]/n_prop[4], n_prop[4]))

  sample.prop = svydesign(id=~CODE, strata = ~clust87, weights = weight_prop,
                          fpc = c(rep(n_prop[1]/N_clust87[1], n_prop[1]),
                                  rep(n_prop[2]/N_clust87[2], n_prop[2]),
                                  rep(n_prop[3]/N_clust87[3], n_prop[3]),
                                  rep(n_prop[4]/N_clust87[4], n_prop[4])),
                          data = prop.data)

  res_prop = svytotal(~ACRES92,sample.prop)
  tu.esti_st[i] <- res_prop[1]
  var.esti.tu_st[i] <- SE(res_prop)^2
}
```

```
#Monte Carlo: mean and standard deviation
Y_MC_st3 = mean(tu.esti_st);Y_MC_st3
V_MC_st3 = var(tu.esti_st);V_MC_st3
SD_MC_st3 = sqrt(var(tu.esti_st));SD_MC_st3
CV_MC_st3 = SD_MC_st3/Y_MC_st3;CV_MC_st3

#Histogram
st3 = data.frame(values = tu.esti_st)
qplot(x = st3$values, fill= ..count.., geom = "histogram", bins = 20) +
  geom_vline(aes(xintercept = T_y, color = "black"), size = 0.5, linetype = "dashed") +
  geom_vline(aes(xintercept = Y_MC_st3, color = "purple"), size = 0.5, linetype = "dashed") +
  xlab("Horvitz Thompson Total estimator") +
  ylab("Count") +
  scale_fill_gradient(low = "black", high = "khaki2") +
  scale_x_continuous(breaks=c(T_y, Y_MC_st3) ,labels=c(" "," ")) +
  scale_color_manual(name = "Statistic",
                     labels=c(expression(italic(t)[italic(y)]),
                              expression(hat(italic(t))[italic(y)])),
                     values=c("black", "magenta")) +
  theme_light(base_family = "Times") +
  theme(legend.text = element_text(size = 12),
        legend.title = element_text(size = 12))
```


#3.3 Post-stratification 1 and 2
```{r PS 1}
# Initialization
set.seed(7)
N_post = table(clust87  = farms$clust87)
nsimu = 1000
nb.simul = matrix(1:nsimu, nsimu, 1)
tu.esti_post = matrix(1, nsimu, 1)
var.esti.tu_post = matrix(1, nsimu, 1)

#Simulations
for (i in 1:nsimu){
  srswor.farms = srswor(n,N)
  design.srswor = svydesign(id = ~CODE, weight = rep(N/n, n),
                        fpc = rep(N, n), data = farms[which(srswor.farms == 1),])
  design.post = postStratify(design.srswor, ~clust87, N_post, partial = TRUE)
  # we use partial = TRUE because in some cases post-strat may be empty
  res.post = svytotal(~ACRES92, design = design.post)
  tu.esti_post[i] <- res.post[1]
  var.esti.tu_post[i] <- res.post[2]
}

#Monte Carlo: mean and standard deviation
Y_MC_post1 = mean(tu.esti_post);Y_MC_post1
V_MC_post1 = var(tu.esti_post);V_MC_post1
SD_MC_post1 = sqrt(var(tu.esti_post));SD_MC_post1
CV_MC_post1 = SD_MC_post1/Y_MC_post1;CV_MC_post1
err_post1 = abs((Y_MC_post1 - T_y)/T_y);print(paste(round(err_post1*100, 3),"%"))

#Histogram
post1 = data.frame(values = tu.esti_post)
qplot(x = post1$values, fill=..count.., geom="histogram", bins = 20) +
  geom_vline(aes(xintercept = T_y, color = "black"), size = 0.5, linetype = "dashed") +
  geom_vline(aes(xintercept = Y_MC_post1, color = "purple"), size = 0.5, linetype = "dashed") +
  xlab("Post Stratified Total estimator 1 (using clust87)") +
  ylab("Count") +
  scale_fill_gradient(low="black", high="royalblue2") +
  scale_x_continuous(breaks=c(T_y, Y_MC_post1) ,labels=c(" "," ")) +
  scale_color_manual(name = "Statistic",
                     labels=c(expression(italic(t)[italic(y)]),
                              expression(hat(italic(t))[post])),
                     values=c("black", "magenta")) +
  theme_light(base_family = "Times") +
  theme(legend.text = element_text(size = 12),
        legend.title = element_text(size = 12))
```

```{r PS2}
# Initialization
set.seed(7)
N_post <- table(REGION  = farms$REGION)
nsimu = 1000
nb.simul = matrix(1:nsimu, nsimu, 1)
tu.esti_post = matrix(1, nsimu, 1)
var.esti.tu_post = matrix(1, nsimu, 1)

#Simulations
for (i in 1:nsimu){
  srswor.farms = srswor(n,N)
  design.srswor = svydesign(id = ~CODE, weight = rep(N/n, n),
                        fpc = rep(N, n), data = farms[which(srswor.farms == 1),])
  design.post = postStratify(design.srswor, ~REGION, N_post, partial = TRUE)
```

```
  # we use partial = TRUE because in some cases post-strat may be empty
  res.post = svytotal(~ACRES92, design = design.post)
  tu.esti_post[i] <- res.post[1]
  var.esti.tu_post[i] <- res.post[2]
}

#Monte Carlo: mean and standard deviation
Y_MC_post2 = mean(tu.esti_post);Y_MC_post2
V_MC_post2 = var(tu.esti_post);V_MC_post2
SD_MC_post2 = sqrt(var(tu.esti_post));SD_MC_post2
CV_MC_post2 = SD_MC_post2/Y_MC_post2;CV_MC_post2
err_post2 = abs((Y_MC_post2 - T_y)/T_y);print(paste(round(err_post2*100, 3),"%"))

#Histogram
post2 = data.frame(values = tu.esti_post)
qplot(x = post2$values, fill=..count.., geom="histogram", bins = 20) +
  geom_vline(aes(xintercept = T_y, color = "black"), size = 0.5, linetype = "dashed") +
  geom_vline(aes(xintercept = Y_MC_post2, color = "purple"), size = 0.5, linetype = "dashed") +
  xlab("Post Stratified Total estimator 2 (using REGION)") +
  ylab("Count") +
  scale_fill_gradient(low="black", high="gold1") +
  scale_x_continuous(breaks=c(T_y, Y_MC_post2) ,labels=c(" "," ")) +
  scale_color_manual(name = "Statistic",
                     labels=c(expression(italic(t)[italic(y)]),
                              expression(hat(italic(t))[post])),
                     values=c("black", "magenta")) +
  theme_light(base_family = "Times") +
  theme(legend.text = element_text(size = 12),
        legend.title = element_text(size = 12))
```


```{r Ratio}
set.seed(7)
nsimu = 1000
nb.simul = matrix(1:nsimu, nsimu, 1)
tu.esti_ratio = matrix(1, nsimu, 1)
var.esti.tu_ratio = matrix(1, nsimu, 1)

for (i in 1:nsimu){
  srswor.farms = srswor(n,N)
  design.srswor = svydesign(id = ~CODE, weight = rep(N/n, n),
                            fpc = rep(N, n), data = farms[which(srswor.farms == 1),])
  est.ratio = svyratio(~ACRES92, ~ACRES87, design.srswor)
  res.ratio = predict(est.ratio, total = sum(ACRES87))

  tu.esti_ratio[i] <- res.ratio[[1]]
  var.esti.tu_ratio[i] <- res.ratio[[2]]
}

Y_MC_ratio = mean(tu.esti_ratio);Y_MC_ratio
V_MC_ratio = var(tu.esti_ratio);V_MC_ratio
SD_MC_ratio = sqrt(V_MC_ratio);SD_MC_ratio
CV_MC_ratio = SD_MC_ratio/Y_MC_ratio;CV_MC_ratio
err_ratio = abs((Y_MC_ratio - T_y)/T_y);print(paste(round(err_ratio*100, 3),"%"))

#Histogram
ratio = data.frame(values = tu.esti_ratio)
qplot(x = ratio$values, fill=..count.., geom="histogram", bins = 20) +
  geom_vline(aes(xintercept = T_y, color = "black"), size = 0.5, linetype = "dashed") +
  geom_vline(aes(xintercept = Y_MC_ratio, color = "purple"), size = 0.5, linetype = "dashed") +
  xlab("Ratio Total estimator") +
  scale_fill_gradient(low="black", high="limegreen") +
  ylab("Count") +
  scale_x_continuous(breaks=c(T_y, Y_MC_ratio) ,labels=c(" "," ")) +
  scale_color_manual(name = "Statistic",
                     labels=c(expression(italic(t)[italic(y)]),
                              expression(hat(italic(t))[rat])),
                     values=c("black", "magenta")) +
  theme_light(base_family = "Times") +
  theme(legend.text = element_text(size = 12),
        legend.title = element_text(size = 12))
```

```{r Regression}
set.seed(7)
nsimu = 1000
nb.simul = matrix(1:nsimu, nsimu, 1)
tu.esti_reg = matrix(1, nsimu, 1)
var.esti.tu_reg = matrix(1, nsimu, 1)

for (i in 1:nsimu){
  srswor.farms = srswor(n,N)
  design.srswor = svydesign(id = ~CODE, weight = rep(N/n, n),
                            fpc = rep(N, n), data = farms[which(srswor.farms == 1),])
  est.reg = calibrate(design.srswor, ~ACRES87, c(N, sum(ACRES87)))
  res.reg = svytotal(~ACRES92, est.reg)
```

```
  tu.esti_reg[i] <- res.reg[1]
  var.esti.tu_reg[i] <- res.reg[2]
}

Y_MC_reg = mean(tu.esti_reg);Y_MC_reg
V_MC_reg = var(tu.esti_reg);V_MC_reg
SD_MC_reg = sqrt(V_MC_reg);SD_MC_reg
CV_MC_reg = SD_MC_reg/Y_MC_reg;CV_MC_reg
err_reg = abs((Y_MC_reg - T_y)/T_y);print(paste(round(err_reg*100, 3),"%"))

#Histogram
reg = data.frame(values = tu.esti_reg)
qplot(x = reg$values, fill=..count.., geom="histogram", bins = 20) +
  geom_vline(aes(xintercept = T_y, color = "black"), size = 0.5, linetype = "dashed") +
  geom_vline(aes(xintercept = Y_MC_reg, color = "purple"), size = 0.5, linetype = "dashed") +
  xlab("Regression Total estimator") +
  ylab("Count") +
  scale_fill_gradient(low="black", high="red") +
  scale_x_continuous(breaks=c(T_y, Y_MC_reg) ,labels=c(" "," ")) +
  scale_color_manual(name = "Statistic",
                     labels=c(expression(italic(t)[italic(y)]),
                              expression(hat(italic(t))[reg])),
                     values=c("black", "magenta")) +
  theme_light(base_family = "Times") +
  theme(legend.text = element_text(size = 12),
        legend.title = element_text(size = 12))
```


```{r Calibration}
set.seed(7)
nsimu = 1000

mar1 = c("clust87",4,762,740,747,743)
mar2 = c("ACRES87", 0, sum(farms$ACRES87),0,0,0)
mar3 = c("ACRES82", 0, sum(farms$ACRES82),0,0,0)

# we don't use state because there are some chances that the sample didn't pick every state
marginmatrix = rbind(mar1, mar2,mar3)
marginmatrix


nb.simul = matrix(1:nsimu, nsimu, 1)
tu.esti_cal = matrix(1, nsimu, 1)
var.esti.tu_cal = matrix(1, nsimu, 1)

for (i in 1:nsimu){
  srswor.farms = srswor(n, N)
  sample = farms[which(srswor.farms==1),]
  calib <- calibration(data = sample ,
                       marginMatrix = marginmatrix,
                       colWeights = "weights",
                       method = "linear",
                       description = FALSE)

  res.cal = weightedTotal(sample$ACRES92, calib)
  tu.esti_cal[i] <- res.cal[1]
  }

Y_MC_cal = mean(tu.esti_cal);Y_MC_cal
V_MC_cal = var(tu.esti_cal);V_MC_cal
SD_MC_cal = sqrt(V_MC_cal);SD_MC_cal
CV_MC_cal = SD_MC_cal/Y_MC_cal;CV_MC_cal
err_cal = abs((Y_MC_cal - T_y)/T_y);print(paste(round(err_cal*100, 3),"%"))

#Histogram
cal = data.frame(values = tu.esti_cal)
qplot(x = cal$values, fill=..count.., geom="histogram", bins = 20) +
  geom_vline(aes(xintercept = T_y, color = "black"), size = 0.5, linetype = "dashed") +
  geom_vline(aes(xintercept = Y_MC_cal, color = "purple"), size = 0.5, linetype = "dashed") +
  xlab("Calibrated Total estimator") +
  ylab("Count") +
  scale_fill_gradient(low="black", high="cyan") +
  scale_x_continuous(breaks=c(T_y, Y_MC_cal) ,labels=c(" "," ")) +
  scale_color_manual(name = "Statistic",
                     labels=c(expression(italic(t)[italic(y)]),
                              expression(hat(italic(t))[cal])),
                     values=c("black", "magenta")) +
  theme_light(base_family = "Times") +
  theme(legend.text = element_text(size = 12),
        legend.title = element_text(size = 12))
```

```{r design comparison}
all = rbind(HT, st1, st2, st3)
all = data.frame(all, Design = c(rep("SRSWOR", nsimu),
```

```
                                    rep("ST1", nsimu),
                                    rep("ST2", nsimu),
                                    rep("ST3", nsimu)))

comparison = ggplot(all) +
   aes(x = values, fill = Design) +
   xlab("Total estimator value") +
   ylab("Density") +
   geom_density(aes(y = ..count..), alpha = 0.5) +
   geom_vline(aes(xintercept = T_y), color = "black", size = 0.5, linetype = "dashed") +
   scale_fill_manual(values = c("grey","lightcoral", "sienna4", "khaki2")) +
   scale_x_continuous(breaks=c(T_y), labels=c(TeX("$t_y$"))) +
   theme_light(base_family = "Times")
comparison
```

```{r design effect}
deff_st_prop87 = V_MC_st1/V_MC_HT
deff_st_neyman = V_MC_st2/V_MC_HT
deff_prop = V_MC_st3/V_MC_HT
deff_st_prop87;deff_st_neyman;deff_prop
```

```{r estimator comparison1}
all = rbind(post1, post2)
all = data.frame(all, Estimator = c(rep("PS1 (clust87)", nsimu),
                                    rep("PS2 (REGION)", nsimu)))

comparison = ggplot(all) +
   aes(x = values, fill = Estimator) +
   xlab("Total estimator value") +
   ylab("Density") +
   geom_density(aes(y = ..count..), alpha = 0.5) +
   geom_vline(aes(xintercept = T_y), color = "black", size = 0.5, linetype = "dashed") +
   scale_fill_manual(values = c("royalblue2","gold1")) +
   scale_x_continuous(breaks=c(T_y), labels=c(TeX("$t_y$"))) +
   theme_light(base_family = "Times")
comparison
```

```{r estimator comparison2}
all = rbind(ratio, reg, cal)
all = data.frame(all, Estimator = c(rep("Ratio", nsimu),
                                    rep("Regression", nsimu),
                                    rep("Calibrated", nsimu)))

comparison = ggplot(all) +
   aes(x = values, fill = Estimator) +
   xlab("Total estimator value") +
   ylab("Density") +
   geom_density(aes(y = ..count..), alpha = 0.5) +
   geom_vline(aes(xintercept = T_y), color = "black", size = 0.6, linetype = "dashed") +
   scale_fill_manual(values = c("cyan", "limegreen", "red")) +
   scale_x_continuous(breaks=c(T_y), labels=c(TeX("$t_y$"))) +
   theme_light(base_family = "Times")
comparison
```

```{r estimator comparison 3}
all = rbind(post1, post2, ratio)
all = data.frame(all, Estimator = c(rep("PS1 (clust87)", nsimu),
                                    rep("PS2 (REGION)", nsimu),
                                    rep("Ratio", nsimu)))

comparison = ggplot(all) +
   aes(x = values, fill = Estimator) +
   xlab("Total estimator value") +
   ylab("Density") +
   geom_density(aes(y = ..count..), alpha = 0.6) +
   geom_vline(aes(xintercept = T_y), color = "black", size = 0.5, linetype = "dashed") +
   scale_fill_manual(values = c("royalblue2","gold1", "limegreen")) +
   scale_x_continuous(breaks=c(T_y), labels=c(TeX("$t_y$"))) +
   theme_light(base_family = "Times")
comparison
```

```{r error and CV comparison}
err_post1
err_post2
err_ratio
err_reg
err_cal

CV_MC_post1
CV_MC_post2
CV_MC_ratio
```

```
CV_MC_reg
CV_MC_cal

#Ratio estimator is the best in terms of accuracy and significance
```